Building Your Own Agent

Business Intelligence & AI — Deep Dive

Kerry Back, Rice University

What Is an Agent?

An agent is a program that:

1. Maintains a conversation with a user
2. Has access to **tools** (functions it can call)
3. **Decides which tools to use** based on the question
4. Chains multiple tool calls before responding

This is different from a script that always runs the same steps. The agent **reasons** about what information it needs and fetches it on the fly.

Why an Agent, Not a Script?

A Script

- Fixed sequence of steps
- Same analysis every time
- Can’t handle follow-up questions
- New analysis = new code

An Agent

- Chooses tools based on context
- Different path for each question
- Conversational — remembers context
- New analysis = just ask

You write the tools once; the agent figures out how to combine them.

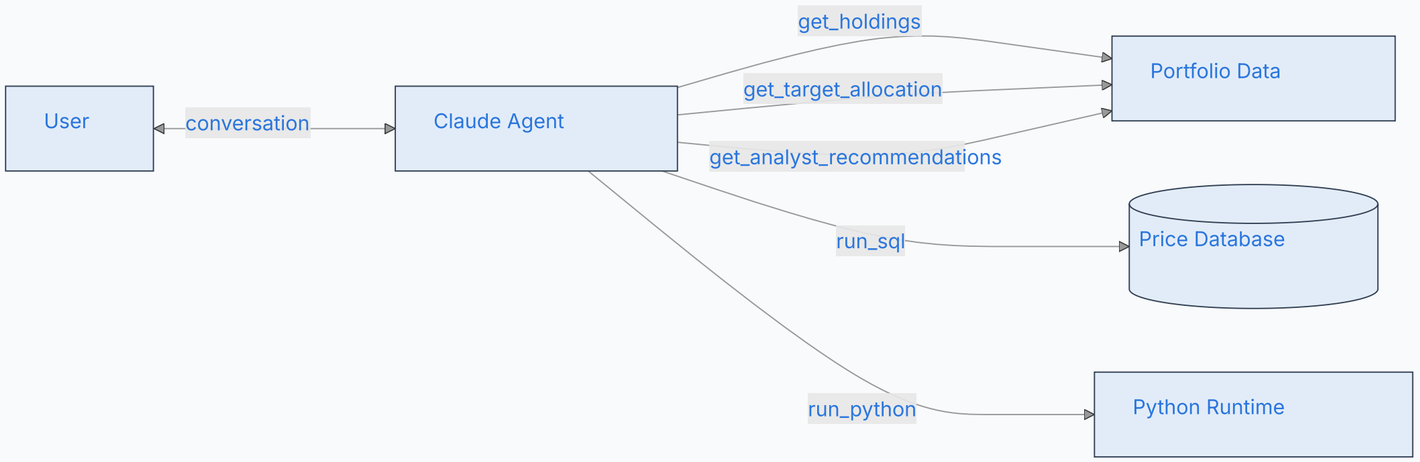
Our Example: Portfolio Review Agent

We'll build a **portfolio analyst** that can:

- Look up current holdings and tax lots
- Query a live price database
- Compute returns, correlations, and statistics in Python
- Compare sector weights against targets
- Identify tax-loss harvesting opportunities
- Find the most correlated Strong Buy replacement

The agent chooses its own path. Asking "review the portfolio" triggers different tools than "find tax-loss candidates in energy."

Architecture



Five tools. Three return portfolio data. One executes SQL against a live price database. One runs Python for analytics.

Step 1: Define the Tools

Tools Are JSON Descriptions

Each tool has a `name`, `description`, and `input_schema`. The description tells the agent **when and why** to use it.

```
tools = [
    {
        "name": "get_holdings",
        "description": (
            "Retrieve all portfolio holdings with tax lot detail. "
            "Returns each position's ticker, sector, and individual "
            "lots (date acquired, shares, cost basis per share)."
        ),
        "input_schema": {
            "type": "object",
```

Tools with no inputs (like `get_holdings`) act as simple lookups. The agent calls them when it needs that data.

The SQL Tool

The most powerful tool lets the agent **write its own queries**:

```
{
    "name": "run_sql",
    "description": (
        "Execute a SQL query against the price database.\n"
        "Available tables:\n"
        "  - sep: daily stock prices (ticker, date, close, closeadj)\n"
        "  - tickers: stock metadata (ticker, name, sector, industry)\n\n"
        "Use closeadj for return calculations."
    ),
    "input_schema": {
```

The agent composes SQL on the fly — correlations, drawdowns, returns — whatever the analysis requires.

The Python Tool

The agent can **compute analytics** on data it retrieves:

```
{
    "name": "run_python",
    "description": (
        "Execute Python code and return printed output. "
        "pandas, numpy, and json are available. "
        "The namespace persists across calls."
    ),
    "input_schema": {
        "type": "object",
        "properties": {
```

Multi-tool chains: `run_sql` fetches prices → `run_python` computes correlations. The agent decides **what** to compute based on the question.

Step 2: Route Tool Calls

The Tool Router

```python
def execute_tool(name: str, inputs: dict) -> str:
    """Route a tool call to the appropriate handler."""
    if name == "get_holdings":
        return json.dumps(HOLDINGS)
    elif name == "get_target_allocation":
        return json.dumps(TARGET_ALLOCATION)
    elif name == "get_analyst_recommendations":
        sector = inputs["sector"]
        recs = STRONG_BUY.get(sector, [])
        return json.dumps({"sector": sector, "strong_buy": recs})
```

This is your code — you control what each tool does. The agent only sees the tool's name, description, and the result you return.

Step 3: The Agent Loop

The Core Pattern

```python
import anthropic

client = anthropic.Anthropic()
messages = []

messages.append({"role": "user", "content": user_input})

while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
```

The loop keeps running until the agent has all the information it needs. It might call one tool or five.

Handling Tool Calls

```
if response.stop_reason == "tool_use":
    # Save the assistant's message
    messages.append({"role": "assistant", "content": response.content})

    # Execute each tool and collect results
    tool_results = []
    for block in response.content:
        if block.type == "tool_use":
            result = execute_tool(block.name, block.input)
            tool_results.append({
```

The `tool_use_id` links each result to the corresponding call. The agent can make **multiple tool calls** in one turn.

The System Prompt

```
SYSTEM_PROMPT = """\
You are a portfolio analyst assistant. You help review a client
portfolio, identify tax-loss harvesting opportunities, and
recommend replacement securities.

When harvesting losses, sell the lots with the largest unrealized
loss first (highest cost basis relative to current price).

When recommending a replacement, compute return correlations
between the stock being sold and each Strong Buy candidate,
```

The system prompt gives the agent its **strategy** — domain expertise that guides tool selection.

Step 4: The Conversation

Example Session

> **You:** Review the portfolio — what's the current sector allocation vs. target?

The agent calls `get_holdings`, then `run_sql` to get current prices, computes market values by sector, and compares to `get_target_allocation`.

> **You:** Find tax-loss harvesting opportunities in Technology.

The agent calls `run_sql` to get current prices for tech holdings, compares against cost basis for each lot, and identifies the biggest losers.

Example Session (continued)

> **You:** Harvest the INTC losses. What should I buy to replace the exposure?

The agent:

1. Identifies INTC lots with the largest unrealized losses
2. Calculates the realized loss
3. Checks if selling moves Technology below target weight
4. Calls `get_analyst_recommendations("Technology")`
5. Calls `run_sql` to fetch daily prices for INTC and each candidate
6. Calls `run_python` to compute return correlations
7. Recommends the most **highly correlated** Strong Buy stock

Seven tool calls across three tools, **none predetermined**. The path depends on what the human asks and what the data shows.

The Agent SDK Simplifies This

Before and After

Manual Loop (~30 lines of plumbing)

```
client = anthropic.Anthropic()
messages = [{"role": "user", "content": prompt}]

while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        system=SYSTEM_PROMPT,
        tools=tools,
        messages=messages,
        max_tokens=4096,
    )
```

Agent SDK (~5 lines)

```
from claude_agent_sdk import query,

async for message in query(
    prompt="Review the portfolio",
    options=ClaudeAgentOptions(
        allowed_tools=[
            "Read", "Edit", "Bash",
        ],
    ),
):
    print(message.result)
```

- **The loop is gone** — the SDK handles it
- Tool execution is automatic
- Message tracking is automatic
- You focus on **what** the agent does, not **how**

Why Both Matter

Understanding the Loop

- You saw the raw machinery
- You know what happens at each step
- You can evaluate vendors and debug failures
- The **checker** needs to understand the system

Using the SDK

- Faster to build
- Less code to maintain
- Fewer places for bugs
- Production-ready patterns built in

The Takeaway

- The manual loop teaches you **what agents are**
- The SDK lets you **build them fast**
- Both use the same tools, same system prompt, same model
- The difference is plumbing, not capability

Key Takeaways

1. **Tools are JSON descriptions** — define what the agent *can* do
2. **The system prompt** — defines what the agent *should* do
3. **The agentic loop** — keep feeding tool results until the agent is done
4. **Human-in-the-loop** — the user steers; the agent executes
5. **Chain tools** — SQL → Python lets the agent fetch data then analyze it
6. **SDKs handle the plumbing** — you focus on tools and strategy

About 200 lines of logic. The complexity lives in the **agent's reasoning**, not in your application code.